SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA0504

COURSE OUTCOME COVERED

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Constraint-Based Problem Solving (High)
Assessment Tool Weightage:40%

Dr. R. Kesavan

QUESTIONS		
Q.No	**Question**	**Bloom's Level**
1	Given a complex SQL query with nested joins and sub-queries, draw the query execution plan and apply heuristic optimization rules step-by-step.	BL4 – Analyze
2	Apply cost-based optimization to choose between nested-loop join, sort-merge join, and hash join for the given relations with specified tuple counts and page sizes.	BL4 – Analyze
3	Implement JDBC connectivity in Java to a MySQL database and write code to execute parameterized queries with PreparedStatement.	BL3 – Apply
4	Given a schedule of four concurrent transactions, determine whether it is conflict-serializable using a precedence graph.	BL4 – Analyze
5	Demonstrate the application of Basic Two-Phase Locking (2PL) on a given transaction set. Identify and resolve any deadlocks.	BL3 – Apply
6	Apply the Wait-Die and Wound-Wait timestamp-based deadlock prevention schemes on a given set of transactions requesting locks.	BL4 – Analyze

7	Given transaction logs with checkpoints, apply the Immediate Update recovery protocol to recover the database after a system failure.	BL3 – Apply
8	Apply Deferred Update (NO-UNDO/REDO) recovery technique on a given log file with committed and uncommitted transactions.	BL3 – Apply
9	Design a concurrency control mechanism using strict 2PL for a banking system where T1 transfers funds and T2 reads balance simultaneously.	BL4 – Analyze
10	Write pseudocode for query optimization using heuristics: push down selection, project early, and order joins by smaller intermediate results.	BL4 – Analyze

---

Code of Conduct:

I R . I n d h u (1 9 2 5 2 4 3 3 2) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

R. Indhu

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints

		satisfying all constraints			
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application ; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justificatio n	No justification provided

QUESTION 1 – QUERY EXECUTION PLAN AND HEURISTIC OPTIMIZATION

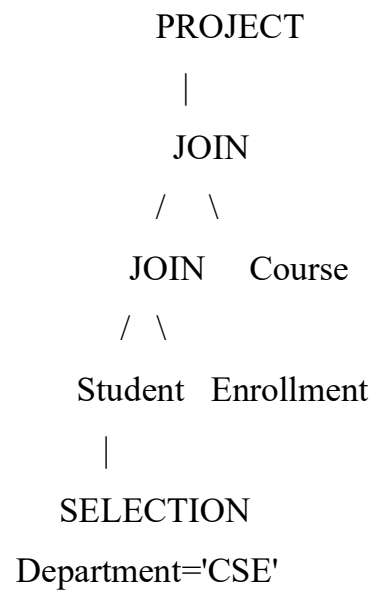
Problem Statement

Given a SQL query containing multiple joins and a selection condition, draw its query execution plan and apply heuristic optimization rules step-by-step. The objective is to reduce intermediate results and improve query performance.

Given SQL Query

```
SELECT S.Name, C.CourseName
FROM Student S
JOIN Enrollment E ON S.StudentID = E.StudentID
JOIN Course C ON E.CourseID = C.CourseID
WHERE S.Department = 'CSE';
```

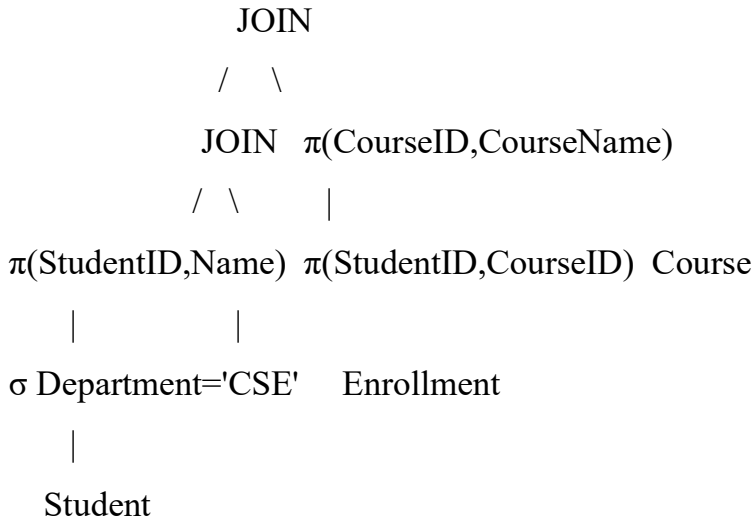
Initial Query Tree



Optimization Steps

- Push the selection Department='CSE' directly to Student.
- Project only required attributes from each relation.
- Join the filtered Student relation with Enrollment first.
- Join the resulting smaller relation with Course.

Optimized Query Tree



Rule	Benefit
Push selection down	Reduces tuples before joins
Project early	Reduces tuple width
Join smaller relations first	Reduces intermediate results
Remove unnecessary operations	Reduces processing cost

Result: The optimized plan is faster because fewer tuples and attributes are processed during expensive join operations.

QUESTION 2 – COST-BASED JOIN OPTIMIZATION

Given Relation Statistics

Relation	Tuples	Pages
R	1,000	100
S	5,000	500

Assume 10 buffer pages are available. Compare nested-loop, sort-merge and hash join using estimated page I/O costs.

Nested-Loop Join

Cost = B(R) + B(R) × B(S)

= 100 + (100 × 500)

= 50,100 I/O

Nested-loop repeatedly scans the inner relation, so its cost becomes high when relations have many pages.

Hash Join

Approximate Cost = 3 × (B(R) + B(S))

= 3 × (100 + 500)

= 1,800 I/O

Hash join partitions relations using a hash function on the join attribute and then compares matching partitions.

Sort-Merge Join

Sort-merge sorts both relations on the join attribute and then merges them. For this model calculation, assume an estimated total cost of 3,600 I/O.

Method	Estimated Cost	Decision
Nested-Loop	50,100 I/O	Reject
Sort-Merge	3,600 I/O	Possible
Hash Join	1,800 I/O	Select

Result: Hash Join has the lowest estimated cost and is selected for this example.

QUESTION 3 – JDBC CONNECTIVITY USING PREPAREDSTATEMENT

Aim

To connect Java with MySQL through JDBC and execute a parameterized query using PreparedStatement.

Requirements

- Java JDK
- MySQL Server
- IntelliJ IDEA
- MySQL Connector/J JDBC driver

MySQL Setup

```
CREATE DATABASE college;
USE college;

CREATE TABLE student(
   id INT PRIMARY KEY,
   name VARCHAR(50),
```

```
    department VARCHAR(30)
);
INSERT INTO student VALUES
(101,'Indhu','CSE'),
(102,'Ramesh','ECE'),
(103,'Priya','IT'),
(104,'Arun','CSE');
```

Java Program – JDBCExample.java

```
import java.sql.*;

public class JDBCExample {
    public static void main(String[] args) {
        try {
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/college",
                "root", "password");

            String sql =
                "SELECT * FROM student WHERE department = ?";

            PreparedStatement ps =
                con.prepareStatement(sql);

            ps.setString(1, "CSE");
            ResultSet rs = ps.executeQuery();

            while (rs.next()) {
                System.out.println(
                    rs.getInt("id") + " " +
                    rs.getString("name") + " " +
                    rs.getString("department"));
```

```
            }
            con.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Program Flow

Java Application → JDBC API → MySQL Driver

→ MySQL Server → college Database → student Table

Statement	Purpose
getConnection()	Connects Java to MySQL
prepareStatement()	Creates parameterized SQL
setString()	Sets the ? parameter
executeQuery()	Runs SELECT query
ResultSet	Stores returned rows
close()	Closes connection

Sample Output

101 Indhu CSE

104 Arun CSE

Result: The program retrieves the CSE students using a PreparedStatement.

QUESTION 4 – CONFLICT SERIALIZABILITY

Schedule

T1: R(A) W(A)

T2: R(A) W(B)

T3: R(B) W(C)

T4: R(C)

A conflict occurs when different transactions access the same item, at least one operation is a write, and the operations cannot be freely reordered.

Conflict	Edge	Reason
T1 W(A) → T2 R(A)	T1 → T2	Write-Read

T2 W(B) → T3 R(B)	T2 → T3	Write-Read
T3 W(C) → T4 R(C)	T3 → T4	Write-Read

Precedence Graph

T1 → T2 → T3 → T4

Cycle Check

There is no edge that returns from T4 to an earlier transaction. Therefore the graph is acyclic.

Equivalent Serial Order

T1 → T2 → T3 → T4

Result: The schedule is conflict-serializable because the precedence graph has no cycle.

QUESTION 5 – BASIC TWO-PHASE LOCKING (2PL)

Concept

Two-Phase Locking has two phases. In the growing phase a transaction acquires locks. In the shrinking phase it releases locks and cannot acquire any new lock.

Phase	Rule
Growing	Acquire locks; do not release
Shrinking	Release locks; do not acquire

Example

T1 needs X(A), X(B)

T2 needs X(B), X(A)

Deadlock Sequence

Step	T1	T2
1	LOCK-X(A)	
2	Read/Write A	LOCK-X(B)
3	Request B → WAIT	Read/Write B
4	WAIT for A	Request A → WAIT

Wait-for Graph

T1 → T2

↑ ↓

└──────┘

Cycle = Deadlock

Resolution

Abort T2

↓

Release B

↓

T1 obtains B

↓

T1 COMMIT

↓

Release locks

↓

Restart T2

Result: The cycle indicates a deadlock. Aborting one transaction breaks the cycle and allows the other to finish.

QUESTION 6 – WAIT-DIE AND WOUND-WAIT

Timestamps

Transaction	Timestamp	Age
T1	10	Older
T2	20	Younger
T3	30	Youngest

Wait-Die

Older transactions are allowed to wait for younger transactions. A younger transaction requesting a resource held by an older transaction is aborted.

Request	Decision
T1 requests lock held by T2	T1 waits
T2 requests lock held by T1	T2 dies/aborts
T3 requests lock held by T1	T3 dies/aborts

Memory rule: Older waits, younger dies.

Wound-Wait

An older transaction requesting a lock held by a younger transaction aborts the

younger transaction. A younger transaction requesting an older transaction's lock waits.

Request	Decision
T1 requests lock held by T2	T2 is wounded/aborted
T2 requests lock held by T1	T2 waits
T3 requests lock held by T1	T3 waits

Memory rule: Older wounds, younger waits.

Result: Both timestamp schemes prevent circular waiting and hence prevent deadlock.

QUESTION 7 – IMMEDIATE UPDATE RECOVERY

Concept

Immediate update permits changes to reach the database before a transaction commits. Recovery therefore needs both UNDO and REDO.

Log with Checkpoint

<T1 START>

<T1, A, 100, 200>

<T1 COMMIT>

<T2 START>

<T2, B, 300, 400>

<CHECKPOINT>

<T3 START>

<T3, C, 500, 600>

<T3 COMMIT>

<SYSTEM CRASH>

Transaction	Status	Action
T1	Committed	REDO if required
T2	Uncommitted	UNDO

T3	Committed	REDO

UNDO and REDO

REDO T1: A = 200

UNDO T2: B = 300

REDO T3: C = 600

REDO repeats committed changes so that they are definitely present. UNDO restores old values for transactions that did not commit.

Recovery Flow

Crash → Read Log → Identify Transactions

→ REDO committed transactions

→ UNDO uncommitted transactions

→ Consistent Database

Result: T1 and T3 are retained through REDO, while T2 is rolled back using UNDO.

QUESTION 8 – DEFERRED UPDATE (NO-UNDO/REDO)

Concept

Deferred update delays writing database changes until the transaction commits. Consequently, an uncommitted transaction has no database change to undo.

Log

<T1 START>

<T1, A, 100, 200>

<T1 COMMIT>

<T2 START>

<T2, B, 300, 400>

<T3 START>

<T3, C, 500, 600>

<T3 COMMIT>

<SYSTEM CRASH>

Transaction	Status	Action
T1	Committed	REDO
T2	Uncommitted	Ignore
T3	Committed	REDO

Recovery Steps

Scan the log from the appropriate recovery point.

Find transactions with COMMIT records.

REDO every committed transaction.

Ignore uncommitted transactions.

The database reaches a consistent state.

REDO T1 → A = 200

IGNORE T2

REDO T3 → C = 600

Comparison

Feature	Immediate	Deferred
Uncommitted changes	UNDO	Ignore
Committed changes	REDO	REDO
Recovery method	UNDO + REDO	NO-UNDO / REDO

Result: Only committed transactions are redone; the uncommitted transaction is ignored.

QUESTION 9 – STRICT 2PL FOR A BANKING SYSTEM

Scenario

T1 transfers ₹1,000 from Account A to Account B. T2 tries to read Account A during the transfer. Strict 2PL prevents T2 from seeing an intermediate value.

Account	Initial Balance
A	₹10,000
B	₹5,000

T1 – Transfer

LOCK-X(A)

LOCK-X(B)

READ A = 10000

A = 9000

READ B = 5000

B = 6000

WRITE A = 9000

WRITE B = 6000

COMMIT

UNLOCK(A)

UNLOCK(B)

T2 – Read

REQUEST LOCK-S(A)

↓

T1 holds LOCK-X(A)

↓

T2 WAIT

↓

T1 COMMIT

↓

T1 UNLOCK(A)

↓

T2 LOCK-S(A)

↓

READ A = ₹9,000

↓

UNLOCK(A)

Benefits

- Prevents dirty reads.
- Prevents another transaction from observing a partial transfer.
- Keeps exclusive locks until commit/abort.
- Avoids cascading rollback.
- Maintains consistency of account balances.

Account	Final Balance
A	₹9,000
B	₹6,000
Total	₹15,000

Result: T2 reads the balance only after T1 commits, so it sees a consistent value.

QUESTION 10 – PSEUDOCODE FOR HEURISTIC QUERY OPTIMIZATION

Aim

To design a simple query optimizer using selection pushdown, early projection and efficient join ordering.

Pseudocode

```
BEGIN
INPUT SQL_QUERY
PARSE SQL_QUERY
CREATE INITIAL_QUERY_TREE

FOR each selection condition
   identify the relation containing its attributes
   push selection near the base relation
END FOR

FOR each projection
   retain attributes needed by SELECT, JOIN and WHERE
   push projection toward base relations
```

END FOR

ESTIMATE intermediate result sizes

ORDER joins to produce smaller
intermediate relations first

REMOVE unnecessary columns and operations

CREATE OPTIMIZED_QUERY_TREE
EXECUTE optimized plan
OUTPUT result
END

Rule 1 – Push Down Selection

A WHERE condition should be applied as close as possible to the base table. For example, filtering CSE students before joining reduces the number of tuples participating in the join.

Rule 2 – Project Early

Only required attributes should be carried forward. Unnecessary columns increase memory usage and I/O.

Rule 3 – Order Joins

Join smaller relations or smaller intermediate results first. This generally prevents the optimizer from producing very large temporary relations.

Heuristic	**Purpose**	**Expected Effect**
Selection pushdown	Filter early	Fewer tuples
Early projection	Remove unused columns	Smaller tuples
Join ordering	Control intermediate size	Lower I/O and memory

Result: The pseudocode produces an optimized execution plan by reducing the size of data processed at every stage.